DOI：10.16661/j.cnki.1672-3791.2305-5042-0756

基于SpringBoot的“喵站”宠物服务平台的设计与实现

艾钰承　朱海风　刘舟
(南京工业大学浦江学院　江苏南京　210000)

摘　要：为了更好地帮助大多数宠物门店服务于顾客，设计并实现了一个基于SpringBoot的宠物服务平台。系统分为管理员端和用户端。管理员可以对已经上架的商品进行修改操作，也可以新增新的商品。用户可以选择自己需要的服务或商品进行购买。在结算时需要添加相关的地址信息，并且可以设置默认地址。该系统可以促进宠物门店的商品的销售，也能为用户提供一个良好的宠物服务平台。

关键词：SpringBoot　宠物服务平台　MVC　MyBatis-Plus
中图分类号：TP311　　**文献标识码：**A　　**文章编号：**1672-3791(2023)22-0022-04

Design and Implementation of the "Miao Zhan" Pet Service Platform Based on Springboot

AI Yucheng　ZHU Haifeng　LIU Zhou
(Pujiang Institute, Nanjing Tech University, Nanjing, Jiangsu Province, 210000 China)

Abstract: In order to better assist most pet stores in serving customers, this article designs and implements a pet service platform based on springboot. The system is divided into the administrator side and the user side. Administrators can edit goods that have already been listed and also add new goods. Users can choose the services or goods they need to purchase, and they need to add relevant address information and can set a default addresses at checkout. This system can promote the sales of goods in pet stores and also provide users with the good pet service platform.

Key Words: SpringBoot; Pet service platform; MVC; MyBatis-Plus

随着我国经济的快速发展、我国居民可支配收入的增长，越来越多的家庭开始饲养宠物，宠物商店、宠物医院和宠物寄养所等宠物服务机构如雨后春笋般出现。为了能使这些机构更好地服务用户，本文设计并实现了一款基于SpringBoot的网络宠物服务平台。

1　系统分析

系统分为用户端和后台管理端。不同的使用者对应不同的功能入口。在用户端，用户需要通过手机验证码进行登录注册，进入用户页面后，用户可以选择自己需要的服务或商品进行购买。在结算时需要添加相关的地址信息，并且可以设置默认地址。后台管理端可以对已经上架的商品进行修改操作，也可以新增新的商品。同时，后台管理端可以通过设置套餐形式将商品捆绑进行销售。在用户下单后，管理端的订单明细页面会更新，并且历史订单会按照时间关系排列，管理员也可以通过输入订单号或时间区间进行查询[1-2]。

2　系统设计与实现

2.1　架构设计

本系统以Springboot为整体框架，采用MVC设计模式，选择MySql为系统数据库[3]。依赖MyBatis-Plus

基金项目：江苏省大学生创新创业训练计划项目资金资助（项目编号：202213905016Y）。
作者简介：艾钰承（2003—），男，本科在读，研究方向为计算机软件工程。
朱海风（2003—），男，本科在读，研究方向为计算机软件工程。
刘舟（2003—），男，本科在读，研究方向为计算机软件工程。

和Druid数据库连接池[4]对数据库进行操作实现对数据的增删改查。

利用Springboot框架，可以快速创建基于spring的程序[5]。使用Springboot框架，不需要人为地进行Tomcat服务器的配置，用户可以直接使用Java main方法启动内嵌的Tomcat服务器运行springboot程序，不需要部署war包[6-7]。同时，Springboot可以根据项目的Maven依赖配置，自动配置spring、springmvc等简化了开发。

MVC包括3个模块：模型层（Model）、视图层（View）、控制层（Controller）。与用户的交互顺序是用户通过视图层发送请求（如查询账户信息request），视图层的请求发送到控制层，控制层进行中转后，通过模型层与数据库交互查询。模型层的查询结果又经由控制层形成响应结果（response），在视图层展示。它的优点是分层设计，可以让各个部分的开发单独开来，减少重复的代码，提高应用程序的可维护性和可扩展性[9]。其架构图如图1所示。

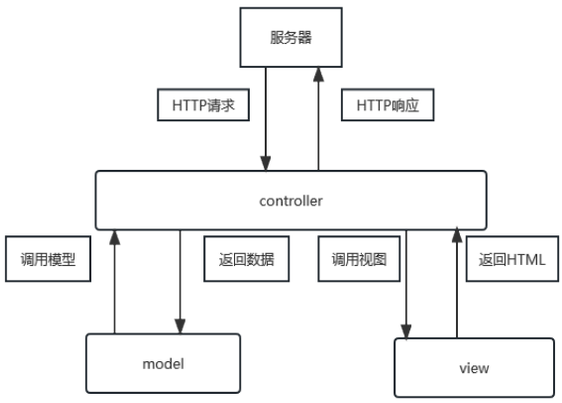

图1 MVC架构图

2.2 功能设计

根据需求分析，本系统分为用户端和后台管理端。用户端功能模块有登录模块、购物模块、添加地址模块、查询历史订单模块；后台管理端的功能模块有新增员工模块、新增服务模块、新增分类模块、新增套餐模块以及查询用户订单模块。系统功能模块如图2所示。

2.3 功能流程设计

2.3.1 用户端功能流程设计

当用户输入手机号，验证码准备登录时，系统会判断验证码输入是否正确。登录成功进入系统后，用户可以选购商品，被选购的商品会进入购物车。当用户完成购物后，提交订单时，系统会判断用户是否设置了默认密码：如果没有设置，页面会自动跳到地址管理页面；如果设置，则可以直接提交订单。

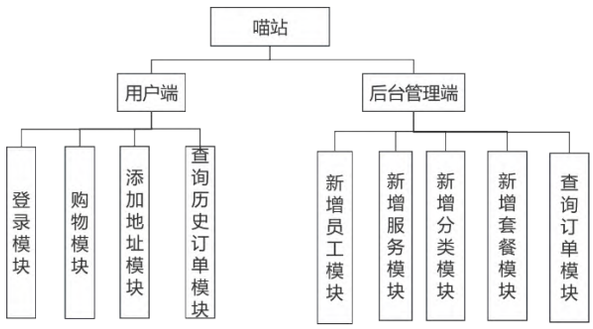

图2 功能模块图

2.3.2 管理员端功能流程设计

当管理员输入用户名和密码后点击登录，系统会进行一次判断，如果两者之中有一个输入错误，就不能成功登录，必须重新输入。当进入系统后，管理员可以进行员工管理，服务信息管理，分类信息管理，套餐信息管理，订单管理的相关方面的操作。

3 关键功能实现

3.1 用户手机验证码登录

该功能逻辑为用户在输入手机号码后收到验证码并输入后才能完成登录，若输入错误后会登录失败。

为了生成验证码，需要创建一个随机验证码生成工具类。该工具类主要通过“Random().nextint()”方法随机生成验证码。选择的是4位验证码，为了防止生成的随机验证码超过四位数需要将“nextint()”中的值定为9 999，这样的话生成的验证码最大数是9 999。同时因为生成的验证码在0~9 999之间，所以有概率生成的验证码是三位数。所以需要进行判断，若生成的验证码小于四位数，需要将该验证码加1 000。相关代码如下：

```
public static Integer generateValidateCode (int length){ Integer code =null; if(length == 4){ code = new Random().nextInt(9999);//生成随机数，最大为9999 if (code < 1000){ code = code + 1000;//保证随机数为4位数字}
return code;}
```

当用户输入手机号码后点击获取验证码后，验证码生成工具类会生成一个四位验证码，通过“session.setAttribute(phone,code);”语句将生成的验证码存入session中。当用户获取到验证码并通过键盘输入后，系统会进行比对：若输入的验证码与session中的验证码相同时，登录成功；若不相同，则登录失败。因为存在新用户的情况，所以需要设置查询语句“Lambda - QueryWrapper<User>queryWrapper = new LambdaQue - ryWrapper<> ( ) ; queryWrapper. eq (User: : getPhone, phone);”通过比对之前数据库中的存储手机号码判断

是否为新用户。若为新用户，则会通过“userService.save(user);”语句自动完成注册。

用户登录页面如图3所示。

图3 用户登录

3.2 地址簿模块

当用户进入“地址管理”页面中，系统先查询存在的地址。首先从session中取出用户id作为查询条件代码为Long userId=(Long) session.getAttribute("user");”。取出后设置泛型为Addrsessbook的查询语句‘LambdaQueryWrapper<AddressBook>queryWrapper =new LambdaQueryWrapper<>();“利用该语句创建queryWapper对象。随后利用之前从session中取出的用户id作为查询条件进行查询，具体代码为”queryWrapper.eq(AddressBook::getUserId,userId);“，这样即可完成地址簿的查询。

如果没有地址，则用户需要进行地址的添加。首先需要设置地址簿的用户id以确保该地址簿绑定在正确的用户上。之后利用IService接口中提供的”save()”方法进行对地址的保存。

当用户需要更改时，利用“addressBookService.updateById(addressBook);”该语句进行对地址的修改；当用户需要删除某个地址时，利用“addressBookService.removeBatchByIds(ids);”进行对地址的删除。更新和删除操作的原理皆是利用了IService接口中的方法，分别为“updateById()”和“removeBatchByIds ()”方法。

为了方便用户在结算时可以不用手动添加地址，可以设置默认地址。首先需要创建查询对象，代码为“LambdaUpdateWrapper<AddressBook> updateWrapper = new LambdaUpdateWrapper<>();”。创建完成后利用当前用户id作为查询条件对地址簿进行查询，代码为“updateWrapper.eq(AddressBook::getUserId,userId);”。因为默认地址有且仅有一个，所以在设置默认地址前需要将所有地址设置为非默认地址。在数据库中，地址的状态被设置为布尔类型，0代表非默认地址，1代表默认地址，所以利用“updateWrapper.set(AddressBook::getIsDefault,0);”该语句把所有地址的状态设置为0。完成以上操作后，通过setIsDefault()方法将用户想要修改的地址的状态设置为1。最后利用updateById()方法将地址簿进行更新。具体代码为：

“addressBook.setIsDefault(1);addressBookService.updateById(addressBook);”。

3.3 新增员工

首先为员工设置默认的登录密码123456。利用setPassword()方法完成添加，其中通过MD5加密对密码加密。具体代码为：

“employee. setPassword (DigestUtils. *md5DigestAsHex*("123456".getBytes()));”。

在完成相关信息填写后，系统调用IService接口中“save()”方法对用户信息进行保存，具体代码为：“employeeService.save(employee);”。

在新增完员工后需要将所有员工进行分页查询后反映到相关页面中。首先建立一个条件构造器，以姓名作为查询条件进行查询，其中需要添加过滤条件，防止姓名为空。具体代码为：

“LambdaQueryWrapper<Employee> queryWrapper = new LambdaQueryWrapper<>();

queryWrapper.like(StringUtils.*isNotEmpty*(name), Employee::getName,name);“

在查询结束后，引入分页构造器，利用员工新增的时间进行降序排序。具体代码为“Page<Employee> pageInfo = new Page<>(page,pageSize);

queryWrapper.orderByDesc(Employee::getUpdateTime);“。

通过以上操作就能完成员工的新增和分页查询。

3.4 新增服务

新增功能主要通过IService接口中提供“save()”方法实现，但是不同的是，一个服务可能有多种配置信息，所以新增服务功能实现了对两张表的插入操作，一张表保存基本信息，另一张表保存服务的配置信息。

保存基本信息利用“save()”方法即可。具体代码为“this.save(serviceDto);”。保存配置信息，需要先创建一个stream流来将服务的id与配置信息进行绑定，使服务与配置信息之间存在关联。完成绑定后，再利用IService接口中提供的”saveBatch()”方法将配置信息进行保存。功能具体代码为：

“Long serviceId = serviceDto.getId();//菜品idList<Flavor> flavors = serviceDto.getFlavors();flavors = flavors.stream().map((item) -> { item.setServiceId(serviceId); return item;}).collect(Collectors.*toList*());ser-

viceFlavorService.saveBatch(flavors);”。

若要删除某个服务，利用IService接口中提供的”removeBatchByIds()”方法进行删除。具体代码为：

”public R<String> delete (String ids) { String [] split = ids.split(","); List<String> list = new ArrayList<> (Arrays.*asList*(split)); serviceService.removeBatchByIds (list); return R.*success*("删除成功！"); }”

在代码中使用了split方法和ArrayList数组将选中的服务id封装成数组，最后通过”removeBatchByIds()”方法可以完成删除操作，这样做既可以删除单个商品，也能进行批量删除。

若要进行更新，仍然是利用IService接口中提供的”updateById()”方法。具体代码为：

“@PutMappingpublic R<String> update (@RequestBody DishDto dishDto) { boolean ret = dishService.updateById(dishDto); return ret? R.*success*("更新成功"):R.*error*("更新失败"); }”。

3.5 新增套餐功能

一个套餐可以包含多个商品，所以若要进行套餐的新增，就要进行两张表的操作。首先通过“save()”方法将套餐的基本信息存入基本信息表中，具体代码为”this.save(setMealDto);

”。接下来创建stream流将新增的套餐id赋予配置信息表中，最后再通过”saveBatch()”方法将配置信息批量保存进配置信息表中。功能具体代码为：

”List<SetMealService>setmealServices = setMealDto.getSetmealServices (); List< SetMealService> collect = setmealServices.stream().map((item) -> { item.setSetmealId (setMealDto. getId ()); return item; }). collect (Collectors. *toList* ()); setMealServiceService. saveBatch (collect);”。

若要删除套餐，先利用”removeByIds()”方法将套餐表中的数据删除。具体代码为“this.removeByIds (ids);”。删掉基本内容后，也需要清理配置信息表中的数据。首先建立一个条件构造器，利用被删除套餐的套餐id作为查询条件将相关配置信息查询出来，最后利用”remove()”方法将相关配置信息删除。具体代码为：

” LambdaQueryWrapper<SetMealService>lambdaQueryWrapper=newLambdaQueryWrapper<> () ; lambdaQueryWrapper.in(SetMealService::getSetmealId,ids); setmealServiceService.remove(lambdaQueryWrapper);”。

3.6 公共字段自动填充

本系统多涉及时间、创建者等方面的问题，如用户注册时的时间、服务更新时的时间、是谁进行了服务的更新等。为了解决这些问题，可以创建一个工具类，每当进行插入和更新操作以及其他操作时，系统会自动记录下这些信息。

为了完成上述功能，关键的是需要让工具类继承“MetaObjectHandler”接口，该接口是MyBatis-Plus提供的一个扩展接口，可以利用这个接口在插入或者更新数据时，为一些字段指定默认值。利用该接口可以快速的实现该功能。当进行插入操作时除了填充创建者和创建时间外，不能让更新者和更新时间为空。所以要同时插入相关的值。具体代码为：

” metaObject. setValue ("createTime", LocalDateTime.*now*()); metaObject.setValue("updateTime", LocalDateTime. *now* ()) ; metaObject. setValue ("createUser", BaseContext.*getCurrentId*()); metaObject.setValue("updateUser", BaseContext.*getCurrentId*());”。

当进行更新操作时，只需要改变更新时间和更新者就行，创建时间和创建者则不需要改变。具体代码为：

” metaObject. setValue ("updateTime", LocalDateTime. *now* ()) ; metaObject. setValue ("updateUser", BaseContext.*getCurrentId*());”。

通过上述操作，系统就能实现公共字段的自动填充的功能。

4 结语

“喵站”宠物综合服务平台成功实现了管理员新增服务、套餐、分类的功能，并且可以通过订单号和时间区间进行对订单的查询。用户端用户也能通过手机验证码进行登录注册并且可以进行对商品的购买、购物车的管理、地址簿的管理，也具有查询历史订单的功能。本平台兼具用户与后台，能较好地满足用户与门店操作方面的需求。

参考文献

[1] 姜丽莉,孙自梅.基于SSM的“C++微课堂”在线实验平台的设计与实现[J].现代信息科技,2022,6(3):36-38.

[2] 郭建卫.基于SpringBoot+Mybatis+Vue的学生德育量化考核系统设计与实现[J].现代信息科技,2023,7(1):18-22.

[3] 李焕,彭科,姜琳琳,等.学生成长档案管理系统的设计与实现[J].现代信息科技,2022,6(3):5-8.

[4] 杨华.基于SSM的宠物店信息系统的设计与实现[D].长春:吉林大学,2019.

[5] 唐苏旭.基于SpringBoot的房屋租赁系统的设计与实现[D].北京:首都经济贸易大学,2021.

[6] 田成龙,陆晓寒,严梦瑶,等.基于SpringBoot的成绩排名系统[J].软件,2021,42(10):129-132.

[7] 苏小伟.基于SpringBoot的汽车维修企业智能化管理系统的设计与实现[D].北京:北京交通大学,2022.